

Assembly Programming

5. Procedures

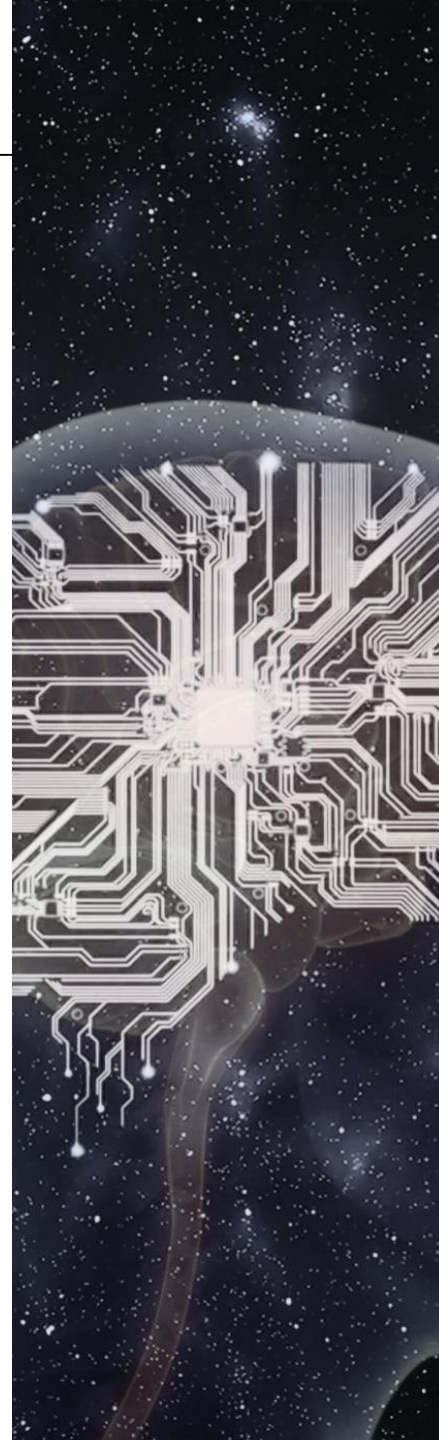
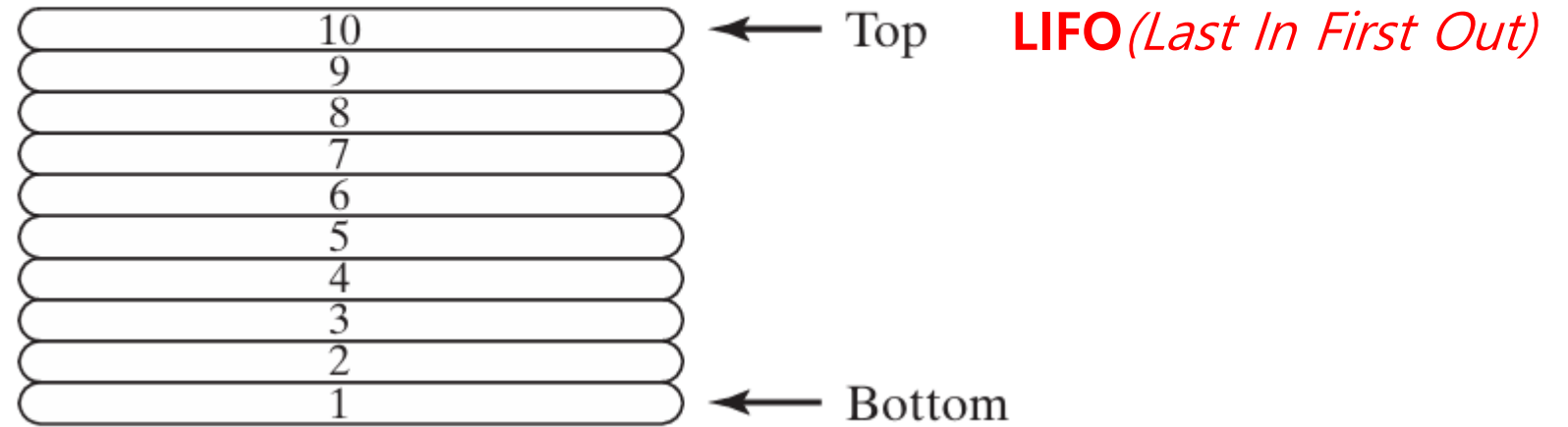


FIGURE 5-1 Stack of plates

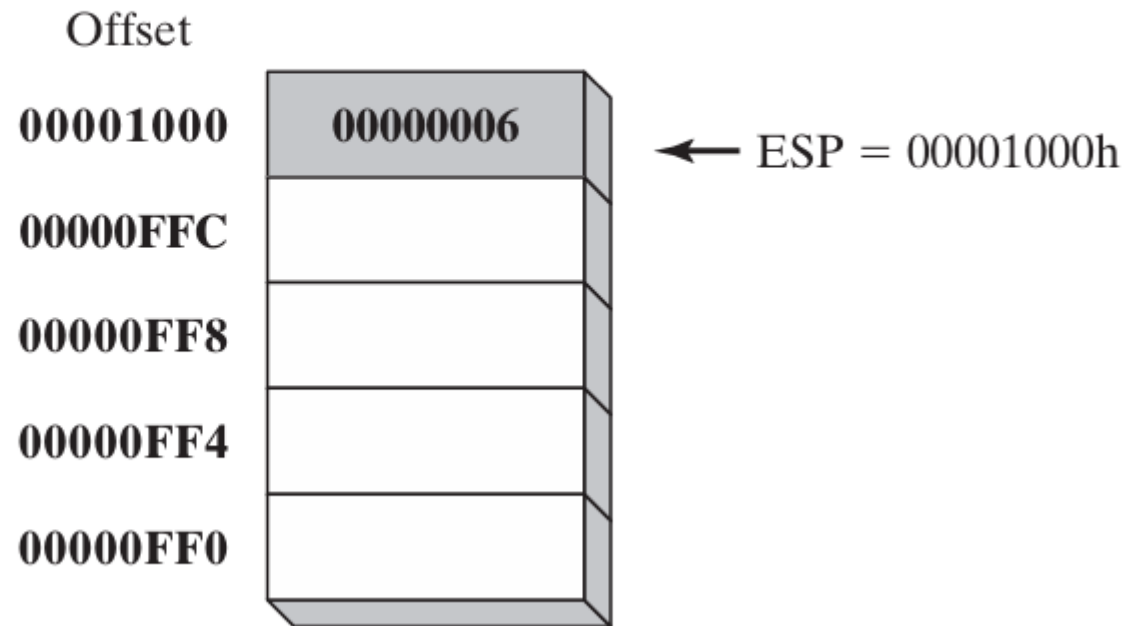


Runtime Stack (32-Bit Mode)

The runtime stack is a memory array managed directly by the CPU, using the ESP (extended stack pointer) register, known as the stack pointer register.

ESP always points to the last value to be added to, or pushed on, the top of stack.

FIGURE 5-2 A stack containing a single value



Stack을 그릴 때는 평소(위 = 낮은 주소, 아래 높은 주소)와 반대로 그린다.

Stack Operations

Push Operation

A 32-bit push operation decrements the stack pointer by 4 and copies a value into the location in the stack pointed to by the stack pointer.

FIGURE 5-3 Pushing integers on the stack.

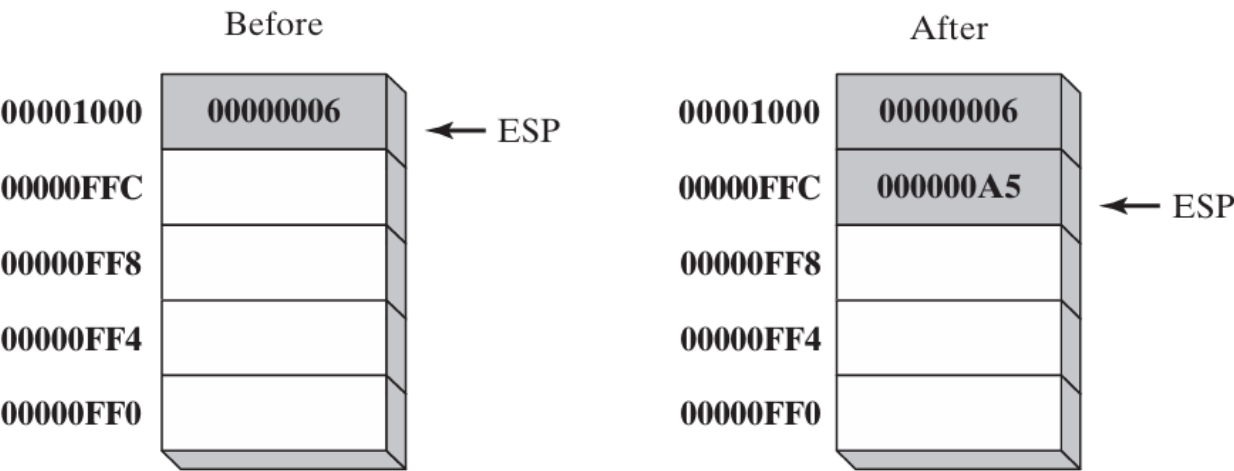
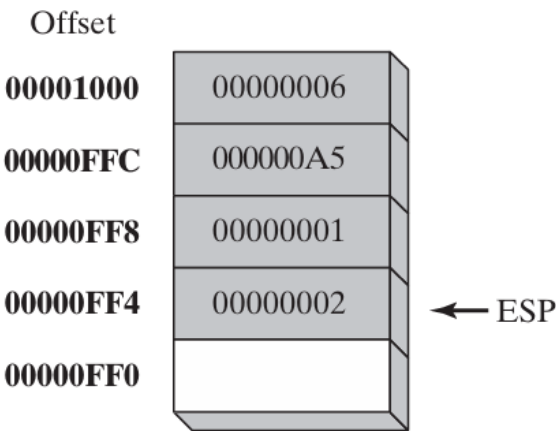


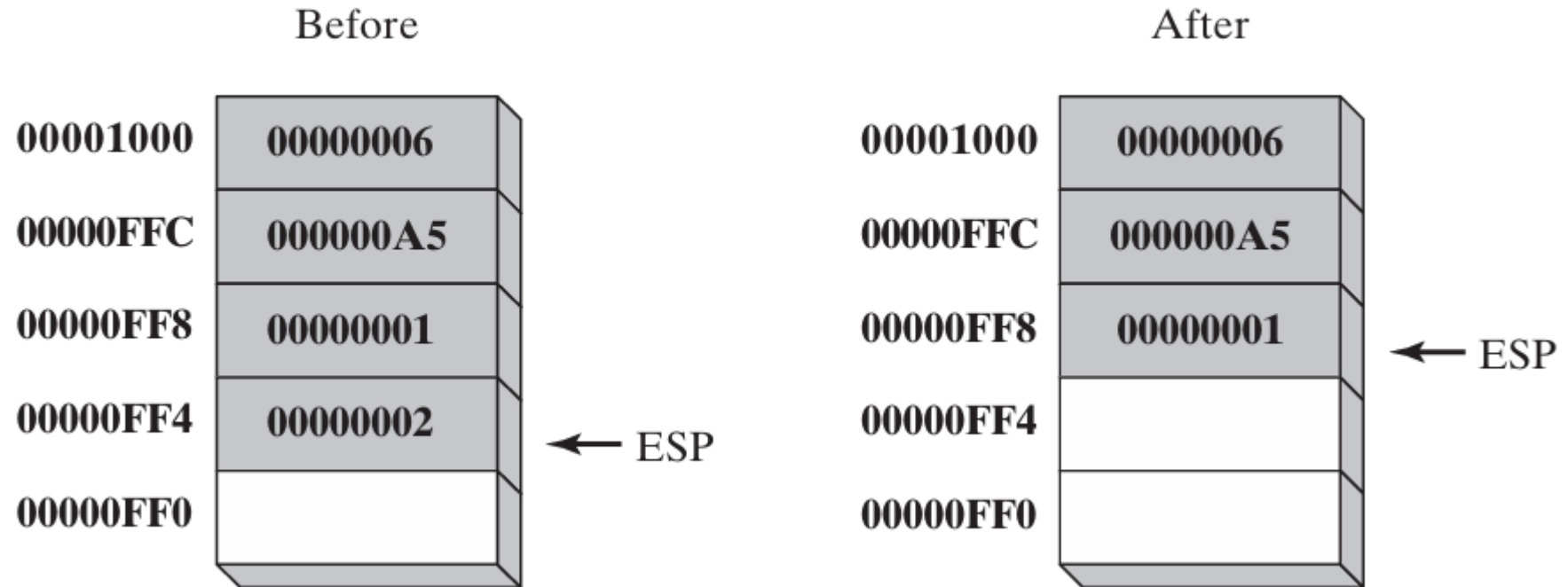
FIGURE 5-4 Stack, after pushing 00000001 and 00000002.



Pop Operation

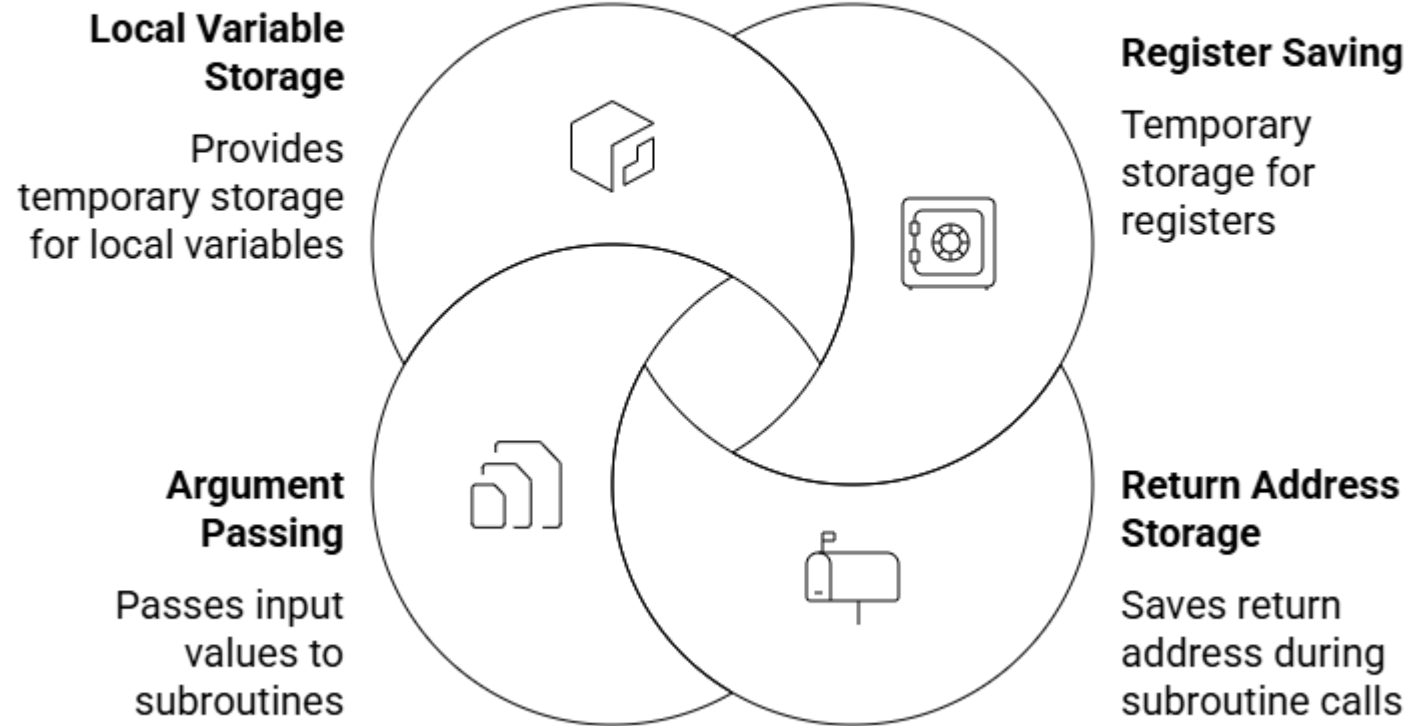
A pop operation removes a value from the stack. After the value is popped from the stack, the stack pointer is incremented (by the stack element size) to point to the next-highest location in the stack.

FIGURE 5–5 Popping a value from the runtime stack.



Stack Applications

Key Uses of Runtime Stacks



Made with  Napkin

PUSH Instruction

The PUSH instruction first decrements ESP and then copies a source operand into the stack.

```
PUSH reg/mem16
```

```
PUSH reg/mem32
```

```
PUSH imm32
```

POP Instruction

The POP instruction first copies the contents of the stack element pointed to by ESP into a 16- or 32-bit destination operand and then increments ESP.

```
POP reg/mem16
```

```
POP reg/mem32
```

PUSHFD and POPFD Instructions

The PUSHFD instruction pushes the 32-bit EFLAGS register on the stack, and POPFD pops the stack into EFLAGS

```
pushfd  
popfd
```

PUSHAD, PUSHA, POPAD, and POPA

The PUSHAD instruction pushes all of the 32-bit general-purpose registers on the stack in the following order: EAX, ECX, EDX, EBX, ESP (value before executing PUSHAD), EBP, ESI, and EDI. The POPAD instruction pops the same registers off the stack in reverse order.

Stack Operations

```
MySub PROC
    pushad                    ; save general-purpose registers
    .
    .
    mov eax,...
    mov edx,...
    mov ecx,...
    .
    .
    popad                    ; restore general-purpose registers
    ret
MySub ENDP
```

```
ReadValue PROC
    pushad                    ; save general-purpose registers
    .
    .
    mov eax,return_value
    .
    .
    popad                    ; overwrites EAX!
    ret
ReadValue ENDP
```

Stack Operations

```
push esi           ; push registers
push ecx
push ebx

mov esi, OFFSET dwordVal ; display some memory
mov ecx, LENGTHOF dwordVal
mov ebx, TYPE dwordVal
call DumpMem

pop ebx            ; restore registers
pop ecx
pop esi
```

```
mov ecx, 100       ; set outer loop count
L1:                ; begin the outer loop
    push ecx       ; save outer loop count

; --- INNER LOOP START ---
    mov ecx, 20    ; set inner loop count
L2:                ; begin the inner loop
    ;
    ;
    loop L2        ; repeat the inner loop
; --- INNER LOOP END ---

    pop ecx        ; restore outer loop count
    loop L1        ; repeat the outer loop
```

Example: Reversing a String

```
.data
aName BYTE "I like Starll",0
nameSize = ($ - aName) - 1           ; nameSize = 12

.code

    mov ecx,nameSize
    mov esi,0
L1:   movzx eax,aName[esi]             ; get character
      push eax                        ; push on stack
      inc esi
      Loop L1

    mov ecx,nameSize
    mov esi,0
L2:   pop eax ; get character
      mov aName[esi],al               ; store in string
      inc esi
      Loop L2
```

Stack Operations

Defining and Using Procedures

PROC Directive

Defining a Procedure subroutines $\hat{=}$ methods or functions

startup procedure main PROC
 .
 .
 main ENDP

sample PROC
 .
 .
 ret
sample ENDP

RET forces the CPU to return to the location from where the procedure was called.

Labels in Procedures

jmp Destination

the label named Destination must be located in the same procedure as the JMP instruction

Destination::
 global label

In terms of program design, it's not a good idea to jump or loop outside of the current procedure.

Defining and Using Procedures

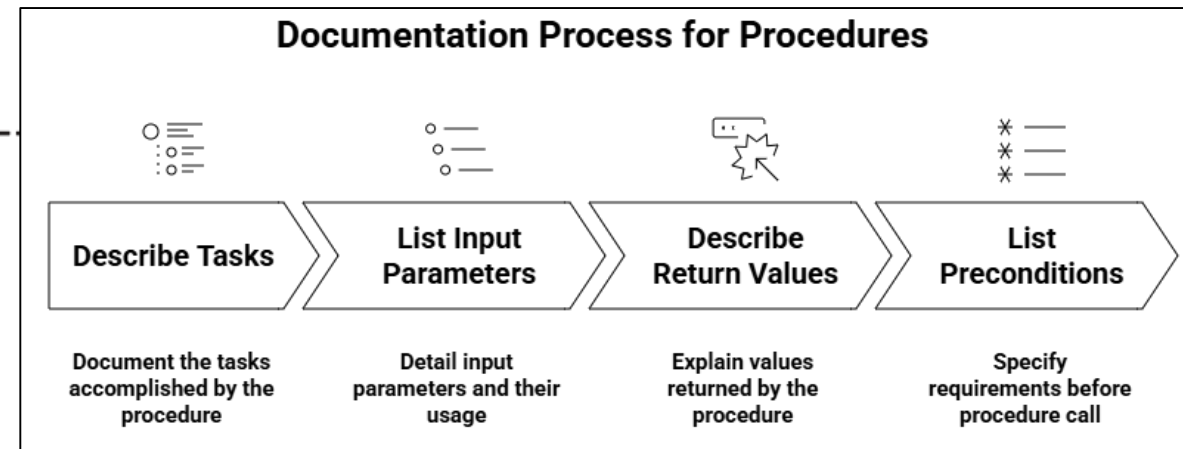
Example: SumOf Three Integers

```
SumOf PROC
    add    eax,ebx
    add    eax,ecx
    ret
SumOf ENDP
```

Documenting Procedures

```
;-----
; sumof
;
; Calculates and returns the sum of three 32-bit integers.
; Receives: EAX, EBX, ECX, the three integers. May be
;           signed or unsigned.
; Returns:  EAX = sum
;-----
```

```
SumOf PROC
    add    eax,ebx
    add    eax,ecx
    ret
SumOf ENDP
```



CALL and RET Instructions

CALL and RET Instructions

What does the CALL instruction do?

It calls a procedure by directing the processor to begin execution at a new memory location.

How does the processor return to the original program point?

The procedure uses a RET (return from procedure) instruction to bring the processor back to the point in the program where the procedure was called.

CALL and RET Instruction Sequence

CALL Instruction Execution

The CALL instruction is executed to call a procedure.

Copy Procedure Address

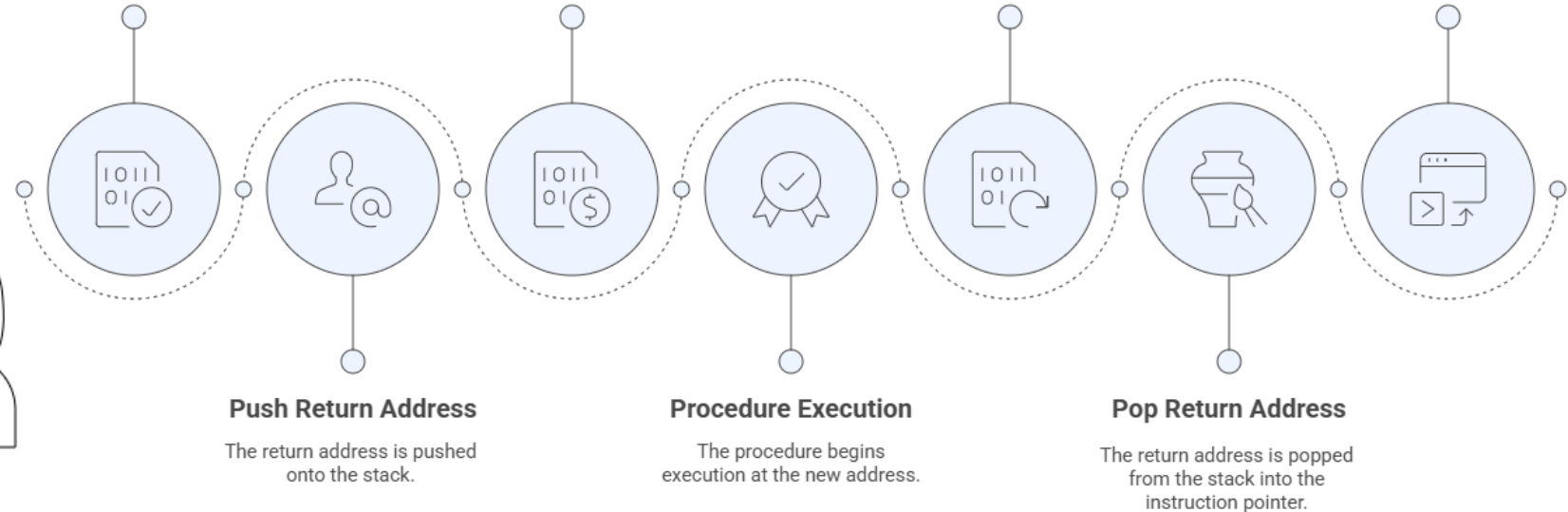
The address of the called procedure is copied into the instruction pointer.

RET Instruction Execution

The RET instruction is executed to return from the procedure.

Continue Program Execution

The program continues execution from the return address.



Call and Return Example

```
main PROC
00000020      call MySub
00000025      mov  eax,ebx
```

```
MySub PROC
00000040      mov  eax,edx
      .
      .
      ret
MySub ENDP
```

FIGURE 5–6 Executing a CALL instruction.

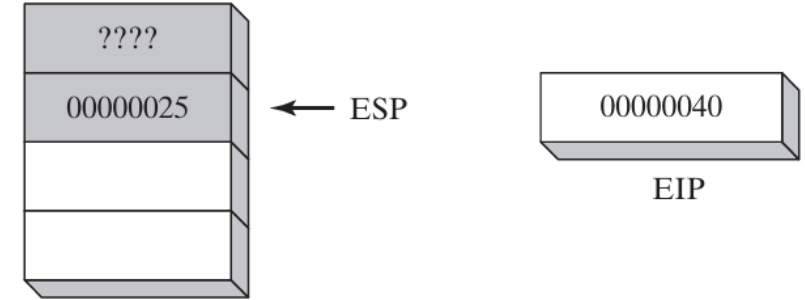
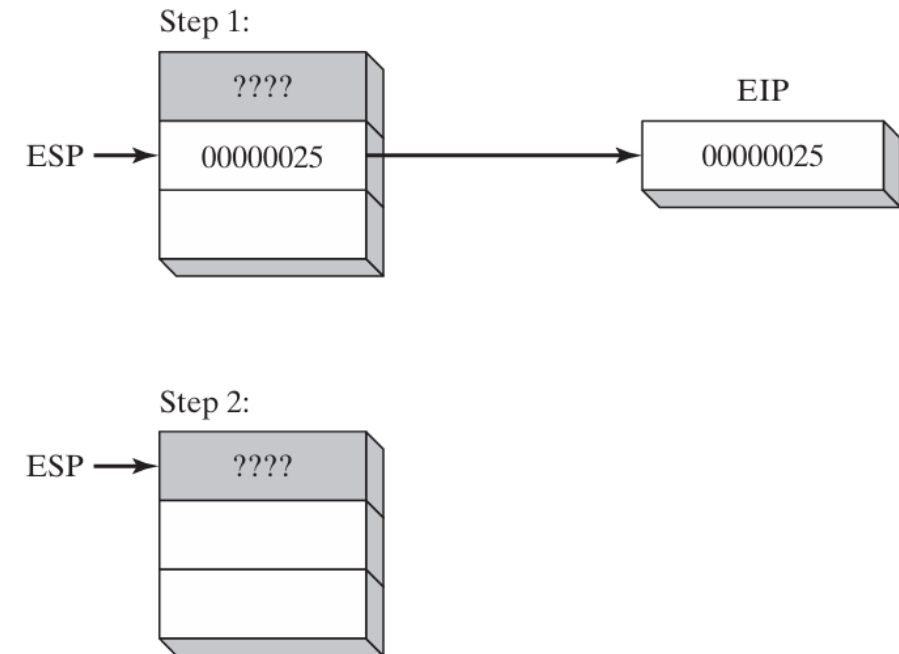


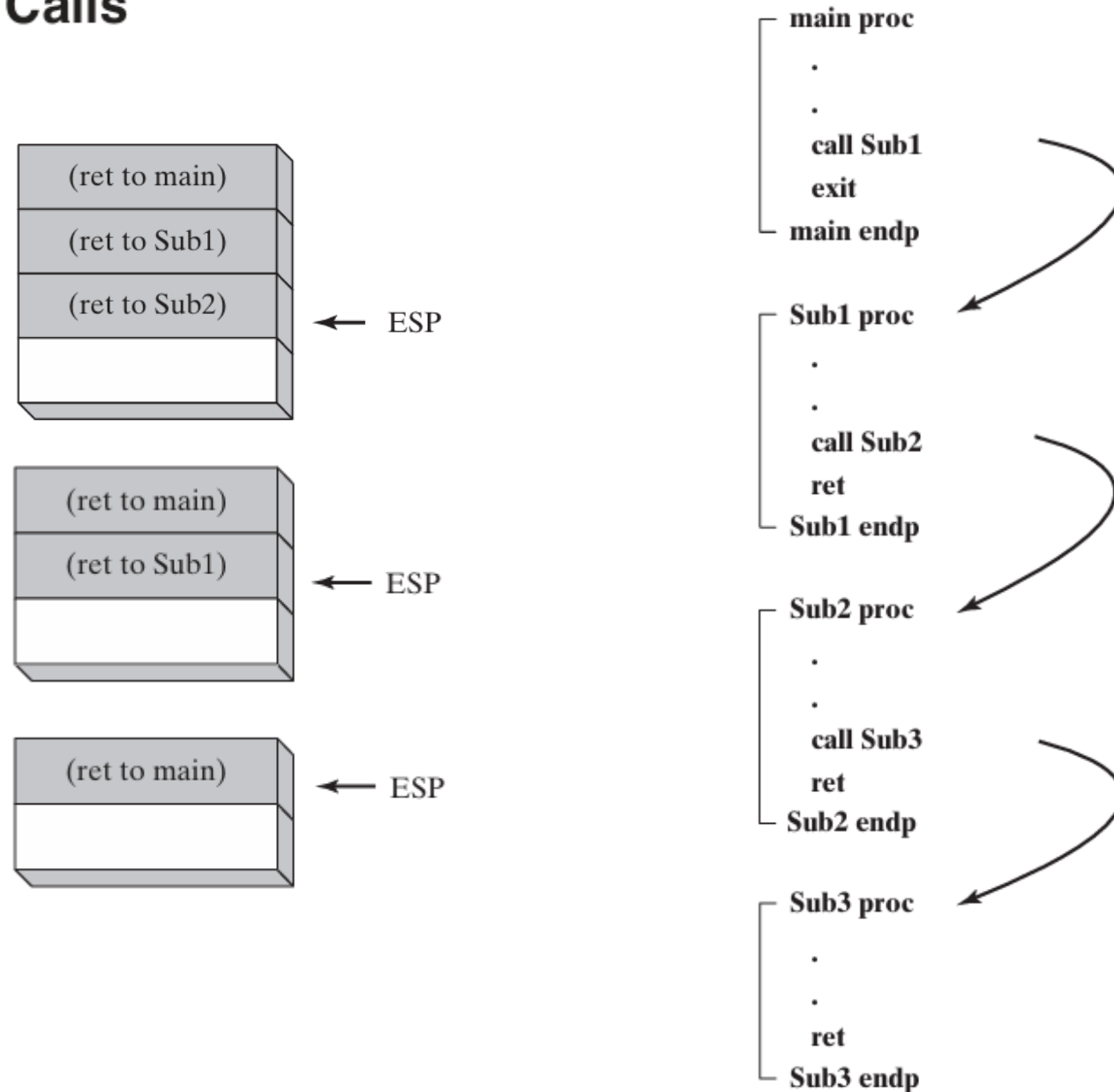
FIGURE 5–7 Executing the RET instruction.



Defining and Using Procedures

Nested Procedure Calls

FIGURE 5-8 Nested procedure calls.



Passing Register Arguments to Procedures

In assembly language, it is common to pass arguments inside general-purpose registers.

```
.data
theSum  DWORD  ?
.code
main PROC
    mov eax,10000h ; argument
    mov ebx,20000h ; argument
    mov ecx,30000h ; argument
    call Sumof      ; EAX = (EAX + EBX + ECX)
    mov theSum,eax  ; save the sum
```

Example: Summing an Integer Array

```
;-----  
; ArraySum  
;  
; Calculates the sum of an array of 32-bit integers.  
; Receives: ESI = the array offset  
;           ECX = number of elements in the array  
; Returns:  EAX = sum of the array elements  
;-----  
ArraySum PROC  
    push esi          ; save ESI, ECX  
    push ecx  
    mov eax,0         ; set the sum to zero  
L1:   add eax,[esi]    ; add each integer to sum  
    add esi,TYPE DWORD ; point to next integer  
    loop L1           ; repeat for array size  
    pop ecx           ; restore ECX, ESI  
    pop esi  
    ret              ; sum is in EAX  
ArraySum ENDP
```

Testing the ArraySum Procedure

```
; Testing the ArraySum procedure (TestArraySum.asm)

.386
.model flat, stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD

.data
array DWORD 10000h,20000h,30000h,40000h,50000h
theSum DWORD ?

.code
main PROC
    mov     esi,OFFSET array           ; ESI points to array
    mov     ecx,LENGTHOF array        ; ECX = array count
    call    ArraySum                  ; calculate the sum
    mov     theSum,eax                 ; returned in EAX
    INVOKE  ExitProcess,0
main ENDP
```

Defining and Using Procedures

```
;-----  
; ArraySum  
; Calculates the sum of an array of 32-bit integers.  
; Receives: ESI = the array offset  
;           ECX = number of elements in the array  
; Returns:  EAX = sum of the array elements  
;-----  
ArraySum PROC  
    push esi                ; save ESI, ECX  
    push ecx  
    mov  eax,0              ; set the sum to zero  
L1:    add  eax,[esi]         ; add each integer to sum  
    add  esi,TYPE DWORD     ; point to next integer  
    loop L1                 ; repeat for array size  
    pop  ecx                ; restore ECX, ESI  
    pop  esi  
    ret                    ; sum is in EAX  
ArraySum ENDP  
END main
```

Saving and Restoring Registers

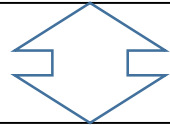
The exception to this rule pertains to registers used as return values, usually EAX.
Do not push and pop them.

USES Operator

USES tells the assembler to do two things:
First, generate PUSH instructions that save the registers on the stack at the beginning of the procedure.
Second, generate POP instructions that restore the register values at the end of the procedure.

Defining and Using Procedures

```
ArraySum PROC USES esi ecx
    mov eax,0                ; set the sum to zero
L1:
    add eax,[esi]             ; add each integer to sum
    add esi,TYPE DWORD        ; point to next integer
    loop L1                   ; repeat for array size
    ret                       ; sum is in EAX
ArraySum ENDP
```



```
ArraySum PROC
    push esi
    push ecx
    mov eax,0                ; set the sum to zero
L1:
    add eax,[esi]             ; add each integer to sum
    add esi,TYPE DWORD        ; point to next integer
    loop L1                   ; repeat for array size
    pop ecx
    pop esi
    ret
ArraySum ENDP
```

Exception

```
SumOf PROC                    ; sum of three integers
    push eax ; save EAX
    add eax,ebx                ; calculate the sum
    add eax,ecx                ; of EAX, EBX, ECX
    pop eax                   ; lost the sum!
    ret
SumOf ENDP
```


Background Information

A **link library** is a file containing procedures (subroutines) that have been assembled into machine code.

A link library begins as one or more source files, which are assembled into object files.

The object files are inserted into a specially formatted file recognized by the linker utility.

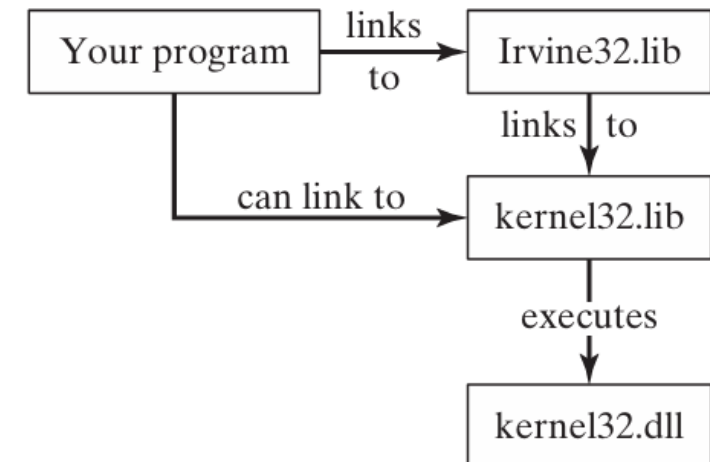
Linker Command Options

```
link hello.obj irvine32.lib kernel32.lib
```

Linking 32-Bit Programs

FIGURE 5–9 Linking 32-bit programs.

The kernel32.lib file, part of the Microsoft Windows Platform Software Development Kit, contains linking information for system functions located in a file named kernel32.dll. The latter is a fundamental part of MS-Windows, and is called a dynamic link library. It contains executable functions that perform character-based input–output.



Linking to an External Library

Motivation for Creating the Library

There is no Microsoft-sanctioned standard library for assembly language programming. In 32-bit mode running under Windows, an input–output library must make calls directly into the operating system. The learning curve is rather steep, and it presents some challenges for beginning programmers.

Therefore, the Irvine32 library is designed to provide a simple interface for input–output for beginners.

Procedures in the Irvine32 Library.

Procedure	Description
CloseFile	Closes a disk file that was previously opened.
Clrscr	Clears the console window and locates the cursor at the upper left corner.
CreateOutputFile	Creates a new disk file for writing in output mode.
Crlf	Writes an end-of-line sequence to the console window.
Delay	Pauses the program execution for a specified n-millisecond interval.
DumpMem	Writes a block of memory to the console window in hexadecimal.
DumpRegs	Displays the EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP, EFLAGS, and EIP registers in hexadecimal. Also displays the most common CPU status flags.
GetCommandTail	Copies the program's command-line arguments (called the command tail) into an array of bytes.
GetDateTime	Gets the current date and time from the system.
GetMaxXY	Gets the number of columns and rows in the console window's buffer.
GetMseconds	Returns the number of milliseconds elapsed since midnight.

The Irvine32 Library

GetTextColor	Returns the active foreground and background text colors in the console window.
Gotoxy	Locates the cursor at a specific row and column in the console window.
IsDigit	Sets the Zero flag if the AL register contains the ASCII code for a decimal digit (0–9).
MsgBox	Displays a popup message box.
MsgBoxAsk	Display a yes/no question in a popup message box.
OpenInputFile	Opens an existing disk file for input.
ParseDecimal32	Converts an unsigned decimal integer string to 32-bit binary.
ParseInteger32	Converts a signed decimal integer string to 32-bit binary.
Random32	Generates a 32-bit pseudorandom integer in the range 0 to FFFFFFFFh.
Randomize	Seeds the random number generator with a unique value.
RandomRange	Generates a pseudorandom integer within a specified range.
ReadChar	Waits for a single character to be typed at the keyboard and returns the character.
ReadDec	Reads an unsigned 32-bit decimal integer from the keyboard, terminated by the Enter key.
ReadFromFile	Reads an input disk file into a buffer.
ReadHex	Reads a 32-bit hexadecimal integer from the keyboard, terminated by the Enter key.

The Irvine32 Library

ReadInt	Reads a 32-bit signed decimal integer from the keyboard, terminated by the Enter key.
ReadKey	Reads a character from the keyboard's input buffer without waiting for input.
ReadString	Reads a string from the keyboard, terminated by the Enter key.
SetTextColor	Sets the foreground and background colors of all subsequent text output to the console.
Str_compare	Compares two strings.
Str_copy	Copies a source string to a destination string.
Str_length	Returns the length of a string in EAX.
Str_trim	Removes unwanted characters from a string.
Str_ucase	Converts a string to uppercase letters.
WaitMsg	Displays a message and waits for a key to be pressed.
WriteBin	Writes an unsigned 32-bit integer to the console window in ASCII binary format.
WriteBinB	Writes a binary integer to the console window in byte, word, or doubleword format.
WriteChar	Writes a single character to the console window.
WriteDec	Writes an unsigned 32-bit integer to the console window in decimal format.
WriteHex	Writes a 32-bit integer to the console window in hexadecimal format.

The Irvine32 Library

WriteHexB	Writes a byte, word, or doubleword integer to the console window in hexadecimal format.
WriteInt	Writes a signed 32-bit integer to the console window in decimal format.
WriteStackFrame	Writes the current procedure's stack frame to the console.
WriteStackFrameName	Writes the current procedure's name and stack frame to the console.
WriteString	Writes a null-terminated string to the console window.
WriteToFile	Writes a buffer to an output file.
WriteWindowsMsg	Displays a string containing the most recent error generated by MS-Windows.

The Irvine32 Library

```
.code
    call Clrscr
    mov eax,500
    call Delay
    call DumpRegs
```

- Clear the screen, delay the program for 500 milliseconds, and dump the registers and flags.

```
.data
    str1 BYTE "Assembly language is easy!",0
.code
    mov edx,OFFSET str1
    call WriteString
    call Crlf
```

```
.data
    str1 BYTE "Assembly language is easy!",0Dh,0Ah,0
.code
    mov edx,OFFSET str1
    call WriteString
```

- Display a null-terminated string and move the cursor to the beginning of the next screen line.

The Irvine32 Library

```
.data
fileName BYTE 80 DUP(0)
.code
    mov edx,OFFSET fileName
    mov ecx,SIZEOF fileName - 1 ; 0 is appended
    call ReadString
```

- Input a string from the user. EDX points to the string and ECX specifies the maximum number of characters the user is permitted to enter.

```
.code
    mov ecx,10                ; loop counter
L1:   mov eax,100              ; ceiling value
      call RandomRange        ; generate random int
      call WriteInt           ; display signed int
      call Crlf               ; goto next display line
      loop L1                 ; repeat loop
```

- Generate and display ten pseudorandom signed integers in the range 0 – 99. Pass each integer to WriteInt in EAX and display it on a separate line.

```
.data
str1 BYTE "Color output is easy!",0
.code
    mov eax,yellow + (blue * 16)
    call SetTextColor
    mov edx,OFFSET str1
    call WriteString
    call Crlf
```

- Display a null-terminated string with yellow characters on a blue background.
- The background color is multiplied by 16 before being added to the foreground color.

Library Test Programs

Tutorial: Library Test #1

```
; Library Test #1: Integer I/O  (InputLoop.asm)
; Tests the Clrscr, Crlf, DumpMem, ReadInt, SetTextColor,
; WaitMsg, WriteBin, WriteHex, and WriteString procedures.

include Irvine32.inc

.data
COUNT = 4
BlueTextOnGray = blue + (lightGray * 16)
DefaultColor = lightGray + (black * 16)
arrayD SDWORD 12345678h,1A4B2000h,3434h,7AB9h
prompt BYTE "Enter a 32-bit signed integer: ",0

.code
main PROC
; Select blue text on a light gray background
    mov eax,BlueTextOnGray
    call SetTextColor
    call Clrscr                ; clear the screen
```

```
    ; Display an array using DumpMem.
    mov esi,OFFSET arrayD                ; starting OFFSET
    mov ebx,TYPE arrayD                  ; doubleword = 4 bytes
    mov ecx,LENGTHOF arrayD              ; number of units in arrayD
    call DumpMem                          ; display memory
    ; Ask the user to input a sequence of signed integers
    call Crlf                            ; new line
    mov ecx,COUNT
L1:  mov edx,OFFSET prompt
    call WriteString
    call ReadInt                          ; input integer into EAX
    call Crlf                            ; new line
    ; Display the integer in decimal, hexadecimal, and binary
    call WriteInt                          ; display in signed decimal
    call Crlf
    call WriteHex                          ; display in hexadecimal
    call Crlf
    call WriteBin                          ; display in binary
    call Crlf
    call Crlf
    Loop L1                              ; repeat the loop
```

```
; Return the console window to default colors
    call WaitMsg                ; "Press any key..."
    mov eax,DefaultColor
    call SetTextColor
    call Clrscr
    exit
main ENDP
END main
```

Library Test #2: Random Integers

```
; Link Library Test #2 (TestLib2.asm)
; Testing the Irvine32 Library procedures.

include Irvine32.inc

TAB = 9                ; ASCII code for Tab

.code
main PROC
    call Randomize      ; init random generator
    call Rand1
    call Rand2
    exit
main ENDP
```

```
Rand1 PROC
; Generate ten pseudo-random integers.
    mov ecx,10          ; loop 10 times
L1:    call Random32     ; generate random int
    call WriteDec        ; write in unsigned decimal
    mov al,TAB           ; horizontal tab
    call WriteChar       ; write the tab
    loop L1
    call Crlf
    ret
Rand1 ENDP
```

```
Rand2 PROC
; Generate ten pseudo-random integers from -50 to +49
    mov ecx,10
; loop 10 times
L1:    mov eax,100        ; values 0-99
        call RandomRange ; generate random int
        sub eax,50        ; values -50 to +49
        call WriteInt     ; write signed decimal
        mov al,TAB        ; horizontal tab
        call WriteChar    ; write the tab
        loop L1
        call Crlf
        ret
Rand2 ENDP
END main
```

Library Test #3: Performance Timing

```
; Link Library Test #3 (TestLib3.asm)
; Calculate the elapsed execution time of a nested loop
```

```
include Irvine32.inc
```

```
.data
OUTER_LOOP_COUNT = 3
startTime DWORD ?
msg1 byte "Please wait...",0dh,0ah,0
msg2 byte "Elapsed milliseconds: ",0
```

```
.code
main PROC
    mov edx,OFFSET msg1    ; "Please wait..."
    call WriteString
; Save the starting time
    call GetMSeconds
    mov startTime,eax
```

```
; Start the outer loop
    mov ecx,OUTER_LOOP_COUNT
L1:    call innerLoop
    loop L1
; Calculate the elapsed time
    call GetMSeconds
    sub eax,startTime
; Display the elapsed time
    mov edx,OFFSET msg2    ; "Elapsed milliseconds:
"
    call WriteString
    call WriteDec          ; write the milliseconds
    call Crlf
    exit
main ENDP
```



```
innerLoop PROC
    push ecx                ; save current ECX value
    mov ecx,0FFFFFFFFh      ; set the loop counter
L1:    mul eax              ; use up some cycles
    mul eax
    mul eax
    loop L1                 ; repeat the inner loop
    pop ecx                ; restore ECX's saved
value
    ret
innerLoop ENDP
END main
```


PROCEDURES

5.1 Stack Operations

- 5.1.1 Runtime Stack (32-Bit Mode)
- 5.1.2 PUSH and POP Instructions
- 5.1.3 Section Review

5.2 Defining and Using Procedures

- 5.2.1 PROC Directive
- 5.2.2 CALL and RET Instructions
- 5.2.3 Nested Procedure Calls
- 5.2.4 Passing Register Arguments to Procedures
- 5.2.5 Example: Summing an Integer Array
- 5.2.6 Saving and Restoring Registers
- 5.2.7 Section Review

5.3 Linking to an External Library

- 5.3.1 Background Information
- 5.3.2 Section Review

5.4 The Irvine32 Library

- 5.4.1 Motivation for Creating the Library

5.4.2 Overview

- 5.4.3 Individual Procedure Descriptions
- 5.4.4 Library Test Programs
- 5.4.5 Section Review

5.5 64-Bit Assembly Programming

- 5.5.1 The *Irvine64* Library
- 5.5.2 Calling 64-Bit Subroutines
- 5.5.3 The x64 Calling Convention
- 5.5.4 Sample Program that Calls a Procedure

5.6 Chapter Summary

5.7 Key Terms

- 5.7.1 Terms
- 5.7.2 Instructions, Operators, and Directives

5.8 Review Questions and Exercises

- 5.8.1 Short Answer
- 5.8.2 Algorithm Workbench

5.9 Programming Exercises

reference

<https://www.asmirvine.com/>

<https://intelligentcm.tistory.com/category/Assembly>

Review questions and Exercises & Programming exercises

Github upload

Presentation subject

- interrupt
- library
- DLL
- DLL 만드는 방법